

Improved Priority Exchange Server: Scientific Contextualization

Milestone 2 — Special Emphasis: Real-Time Systems, HSHL

Sumon Boiddo

BEng in Electronic Engineering

Hochschule Hamm-Lippstadt

Matriculation Number: 2220054

Group B, Team 6

Abstract—This document is the Milestone 2 (Scientific Contextualization) for the seminar topic *Improved Priority Exchange Server (IPE)*. It places IPE within the landscape of aperiodic servers, compares it with its neighbours (PE, DPE, EDL, and TBS) through a comparison table and detailed pairwise comparisons, analyses its scientific and engineering trade-offs, discusses its consequences for embedded systems, and explains which related work is excluded and why. The primary reference is Buttazzo, *Hard Real-Time Computing Systems* [1], Chapter 6.

I. OVERVIEW OF THE APERIODIC SERVER LANDSCAPE

- Aperiodic servers are used to serve soft aperiodic tasks together with hard periodic tasks. They are grouped into two families, based on the scheduler used for the periodic tasks [1].
- The *fixed-priority* (Rate Monotonic, RM) family uses fixed priorities. It includes the Polling Server (PS), the Deferrable Server (DS), the Priority Exchange (PE) server, and the Sporadic Server (SS).
- The *dynamic-priority* (Earliest Deadline First, EDF) family uses deadlines to decide priority. It includes the Dynamic Priority Exchange (DPE) server, the Dynamic Sporadic Server (DSS), the Total Bandwidth Server (TBS), the Earliest Deadline Late (EDL) server, and the Improved Priority Exchange (IPE) server.
- IPE belongs to the dynamic-priority (EDF) family. Under EDF the periodic utilization bound is 100%, compared with about 69% for RM with many tasks, which allows larger servers and better aperiodic responsiveness [1].
- IPE comes from a clear family tree: the fixed-priority PE server was first extended to EDF as the DPE server, and DPE was then improved into IPE by using the precomputed idle times of the EDL server. The lineage is therefore PE → DPE → IPE [3].

II. COMPARISON OF APERIODIC SERVER APPROACHES

Table I compares IPE with its four closest neighbours — PE, DPE, EDL, and TBS — across three aspects: the scheduler each one uses, its responsiveness (how quickly it serves aperiodic requests), and its main limitation. The table highlights three points. First, only PE uses fixed priority, while

all the other servers use EDF. Second, EDL is optimal and IPE is nearly optimal, so these two give the best responsiveness. Third, each server pays a different price: EDL pays in heavy run-time computation, IPE pays in memory, and TBS accepts a small loss of optimality in exchange for simplicity.

III. DETAILED COMPARISON: IPE VS. NEIGHBOURING SERVERS

IPE vs. PE. PE is the fixed-priority (RM) ancestor of IPE. IPE moves this idea to EDF and adds the precomputed EDL idle times. As a result, IPE serves aperiodic tasks much faster (nearly optimal), whereas PE, being fixed-priority, is less responsive [3].

IPE vs. DPE. DPE is IPE's direct base; both use EDF and the same priority-exchange mechanism. The improvement lies in the replenishment policy: DPE uses a periodic refill, while IPE refills using the EDL idle times precomputed offline (the E and D arrays) and always runs at the highest priority. This makes IPE more efficient and more responsive than DPE [3].

IPE vs. EDL. EDL is the optimal server (minimum aperiodic response time), and IPE nearly matches it. The key difference is *when* the computation is performed: EDL recomputes the schedule at run time (online), which is expensive, while IPE precomputes the same idle-time information offline, which is far cheaper at run time. IPE therefore achieves nearly the same responsiveness as EDL at a much lower run-time cost, paying instead in memory [1].

IPE vs. TBS. TBS is another EDF server, but it is much simpler: it assigns each aperiodic request a deadline and schedules it directly under EDF, with almost no overhead and no stored tables. IPE is more complex, requiring the precomputed E and D arrays and the priority-exchange bookkeeping, so it needs more memory. The trade-off is that TBS is lighter and simpler but not optimal, while IPE is nearly optimal — especially under heavy aperiodic load — at the cost of offline precomputation and additional memory [1].

IV. SCIENTIFIC AND ENGINEERING TRADE-OFFS

- **Optimality vs. practicality.** The EDL server is optimal but impractical, because it recomputes the schedule at

TABLE I
COMPARISON OF APERIODIC SERVER APPROACHES

Server	Scheduler	Responsiveness (speed)	Main limitation
PE (Priority Exchange)	RM (fixed)	Moderate	Fixed-priority only; less responsive than the EDF servers
DPE (Dynamic Priority Exchange)	EDF	Good	Not optimal; replenishment less efficient than IPE
EDL (Earliest Deadline Late)	EDF	Optimal (best)	Heavy run-time calculation; too expensive to use in practice
TBS (Total Bandwidth Server)	EDF	Very good	Not optimal; but very simple and lightweight
IPE (Improved Priority Exchange)	EDF	Nearly optimal	High memory cost (the stored tables become very large when the periods are non-harmonic)

run time. IPE gives up a very small amount of optimality (it is *nearly* optimal instead of optimal) in exchange for becoming practical, since its heavy work is done offline. This is the central trade-off that defines IPE.

- **Run-time cost vs. memory cost.** IPE moves the heavy idle-time calculation from run time to an offline step. This saves run-time cost but must be paid for in memory: the precomputed E and D arrays grow with the hyperperiod and become very large when the periods are non-harmonic.
- **Performance vs. flexibility.** Because everything is pre-computed for a fixed, known task set, IPE performs very well on static systems but cannot adapt to dynamic systems. If the task set changes at run time, the precomputed plan becomes invalid and the guarantees are lost. IPE thus trades flexibility for high performance.
- **Near-optimal performance vs. simplicity.** Compared with TBS, IPE is nearly optimal, especially under heavy load, but it is more complex (precomputed tables plus priority-exchange bookkeeping). TBS is much simpler and lighter but not optimal. IPE therefore trades simplicity for higher responsiveness.

V. EMBEDDED SYSTEMS CONSEQUENCES

- To deploy IPE, the operating system must provide an EDF scheduler, and an offline analysis step must compute and store the E and D arrays before the system starts running. The heavy idle-time computation is therefore a design-time activity, not a run-time one.
- IPE requires enough memory to store the E and D tables, whose size depends on the hyperperiod $H = \text{lcm}(T_1, \dots, T_n)$. Since embedded devices often have limited memory, this is a critical practical constraint.
- **Consequence for small devices:** IPE fits well when the periods are harmonic (small hyperperiod, small tables, little memory). But when the periods are non-harmonic, the tables become very large and may not fit in the limited memory of a small device, which makes IPE unsuitable there.
- **Overall:** IPE is best suited to static embedded systems with a fixed, known task set, harmonic periods, an EDF-capable operating system, and a need for fast aperi-

odic response while guaranteeing all hard deadlines. It is unsuitable for dynamic systems (changing task set), non-harmonic period sets (memory too large), or fixed-priority-only platforms.

VI. WHY SOME RELATED WORK IS EXCLUDED

- This briefing focuses only on the servers in IPE's own family (PE, DPE, EDL, TBS). A few related approaches are mentioned but not studied in depth:
- The *Constant Bandwidth Server (CBS)* is not part of the IPE topic and is a separate seminar subject; it also follows a different design idea (bandwidth reservation, not priority exchange). For these reasons it is excluded from deep study here.
- The *Dynamic Sporadic Server (DSS)* lies outside the priority-exchange lineage of IPE and is referenced only as a performance comparison point.
- *Multiprocessor scheduling and shared-resource protocols* are covered in separate chapters and fall outside the single-processor, fully-preemptive scope assumed for IPE.

REFERENCES

- [1] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3rd ed. Springer, 2011, ch. 6 (Dynamic Priority Servers).
- [2] M. Spuri and G. Buttazzo, "Efficient Aperiodic Service under Earliest Deadline Scheduling," in *Proc. IEEE Real-Time Systems Symposium (RTSS)*, 1994.
- [3] M. Spuri and G. Buttazzo, "Scheduling Aperiodic Tasks in Dynamic Priority Systems," *Real-Time Systems*, vol. 10, no. 2, March 1996.
- [4] J. Lehoczky, L. Sha, and J. Strosnider, "Enhanced Aperiodic Responsiveness in Hard Real-Time Environments," in *Proc. IEEE Real-Time Systems Symposium (RTSS)*, 1987.
- [5] H. Chetto and M. Chetto, "Some Results of the Earliest Deadline Scheduling Algorithm," *IEEE Trans. Software Engineering*, vol. 15, no. 10, 1989.
- [6] S. Boiddo, *AI Usage Protocol, Milestone 2: Improved Priority Exchange Server*, HSHL, 2026. Protocol version 0.2; JSON and BibTeX files submitted with this milestone.